Lec-31

07/08/26 10:21 AM

Bit masking

0/1 → variable

$\frac{7}{2} \times 1 = 3$

a	b	a&b	a|b	a^b
0	0	0	0	0
1	0	0	1	1
0	1	0	1	1
1	1	1	1	0

① left shift
② Right shift

&& → logic and
& → Bitwise and
|| → logic or
| → Bit or
^ → XOR

!(F) = T

~(0) = 1
~(1) = 0

$(\frac{7}{2}) \times 2 = 6$

%

$(\frac{n}{2}) \times 2 == n$

N = 79

even

odd

$2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$ → 1 ① (1)

0 → even

2^{31}

[2, 3, 1, 1, 2, 3] ①

n%2 == 0

n&1

000000 0/1

2	13	
2	6	1
2	3	0
2	1	1
	0	1

110①?

n&1

& 1101
0001
00001

a^a = 0

5^5
101
101
000

5^7^5 = 7
5^5^7 = 7
7^5^5 = 7

a^b = 0^1 = 1
1^0 = 1
0^0 = 0
1^1 = 0

0^a = a

5
101
000
101

Left-Shift

7 << 1 = 7×2 = 14

7 << 2 = 7×2^2 = 28

n << b ⇒ $n \times 2^b$

N = 13

00001101

00

0 0001101
0001101 0 → 26

-1 → 31
-2 → 30
-3 → 29

(-1) + 32 = (31)
-2 + 32 = 30

n1 << 289

289
n1 × 2

```
public static void main(String[] args) {
	// TODO Auto-generated method stub
	int n1=13;
	int n2=-13;
	System.out.println(n1<<2);
	System.out.println(n2<<2);

    }
```

n1 << -2

n1 << x

x to 0 to 31

x % 32

0 ------ 31

32 → 0

Right Shift

13 >> 1 = 6

16 >> 2 = $\frac{16}{2^2}$ = 4

n >> b = $\frac{n}{2^b}$

0000110~~1~~ → 0000110

6

11110010
1
11110011 → -13

111 11001
00000110
1
0 111

-13/2 = (7)

T.C. log(n)

while(n>0){
if((n&1)!=0)
c++
n>>=1
}

10
100
1000
10

---

① N = 84

2	84	0
2	42	0
2	21	1
2	10	0
2	5	1
2	2	0
2	1	1
	0	

4th → 0 → unset bit
1 → set bit

01010100
43210

Right shift

1<<4
10
100
1000
10000

& 01010100
00010000

mask = 1<<i

if((n&1)==0){
}
if(n%2==1){
c++
}

n = n/2

n >> 1

& 01010100
01010011
01010000
01001111
01000000
& 00111111

01000000 (7)

while(n>0){
n=n&(n-1)
c++
}

n = (n&(n-1))

(-4)

$(-2)^2$ = (4)

00000100
11111011
1
11111100

0001000
1110111
1
1111000

0 1 0
1 0 1
1 1 1

[2, 3, 5, 7, 2, 3, 11, 2, 3, 5, 7]

	u1	u2
23	5 7 23 5 7	2 3 2 3 11

(11)

^ 0001011
0101111
0011100 0

n&(-n)

00011100
11100011
1

111 00100 -n
& 00011100
000000100 → mask

100

(mask & arr[i]) == 0

arr = 0

a^2 ^3 ^2 ^3 ^11

(11)

The problem is:

You are given a *lucky number n*. *Lucky numbers* are the positive integers whose decimal representations contain only the lucky digits 4 and 7. For example, numbers **47**, **744**, **4** are lucky and **5**, **17**, **467** are not.

If we sort all lucky numbers in increasing order, what's the 1-based index of *n*?

Tavas is not as smart as SaDDas, so he asked you to do him a favor and solve this problem so he can have his headphones back.

Input
The first and only line of input contains a lucky number n ($1 \le n \le 10^9$).

Output
Print the index of n among all lucky numbers.

474774

4	7	44	47	74	77	444	447	474	477	744	747	774	777
1	2	3	4	5	6	7	8	9	10	11	12	12	13

474774
n

x+y+1

4/7
22 2

1 → 2
2 → 2^2
3 → 2^3
4 → 2^4

1<<n

1×2^n

$(2^1\ 2^2\ 2^3\ 2^4 \ldots 2^{n-1})$ x + ndigit < y

$2\frac{[2^{n-1}-1]}{2-1} = 2 \times 2^{n-1} - 2 = (2^n - 2)$ x

④74774
-4

pos = 2

(2)
2 + 4 4 6

(22)

$2^6 - 2$ = 62 + 22

= (84 + 1) (85)

A magic number is defined as a number which can be expressed as a power of 5 or sum of unique powers of 5. First few magic numbers are 5, 25, 30(5 + 25), 125, 130(125 + 5), ...

Write a function to find the nth Magic number.

Example:

```
Input: n = 2
Output: 25
Input: n = 5
Output: 130
```

1	2	3	4	5	6	7	8	9	10	11
5	25	30	125	130	150	155	625	630	(650)	655
5^1	5^2	$5^2 5^1$	5^3	$5^3 5^1$	$5^3 5^2$	$5^3 5^2 5^1$	5^4	$5^4 5^1$	$5^4 5^2$	$5^4 5^2 5^1$

1010
$5^4\ 5^3\ 5^2\ 5^1$

2	10	0 × 5^1
2	5	1 × 5^2
2	2	0 × 5^3
2	1	1 × 5^4
	0	

mul = 5

while(n>0){
if((n&1)!=0)
ans = ans + mul
mul = mul × 5
n >>= 1

2^3 → 8

abc

ac

000 → ∅
001 → a
010 → b
011 → ab
100 → c
101 → ac
110 → bc
111 → abc

i → ∟

5
10① x
010 x
00①